VIT Vidyalankar Institute of Technology

End Semester Examination (MU)

Evaluation of University Question Paper Solution Prepared by Faculty

Subject (Write in full) : DATA STRUCTURE (Regular / KT)

Exam : May 2025 / ~~Nov. 20~~ Exam Date : 16/05/2025 **Q. Paper Code :** _____

Department : CMPN **Year : FE/SE/TE/BE/ME/MMS Semester :** 2 **Scheme : CBSGS/CBCS**

Handwritten Solution Prepared by : Prof. Swapnil S. Sonawane

Name of the Subject Cluster : Algorithm Analysis and Design.

Name of the Cluster Mentor / Assessor : Dr. Sanjeev Dwivedi.

1st Assessment :

Question No.	Marks Obtained						Total
1	(a) 5	(b) 5	(c) 5	(d)	(e)	(f)	15
2	(a) 5	(b) 5	(c) 5	(d)	(e)	(f)	15
3	(a) 10	(b) 10	(c) ~~00~~	(d)	(e)	(f)	20
4	(a) 10	(b) 10	(c) ~~00~~	(d)	(e)	(f)	20
5	(a)	(b)	(c)	(d)	(e)	(f)	
6	(a)	(b)	(c)	(d)	(e)	(f)	
Total (Out of marks)							70/70

Signature of Assessor / Cluster Mentor : Sanjeev

2nd Assessment :

Question No.	Marks Obtained						Total
1	(a)	(b)	(c)	(d)	(e)	(f)	
2	(a)	(b)	(c)	(d)	(e)	(f)	
3	(a)	(b)	(c)	(d)	(e)	(f)	
4	(a)	(b)	(c)	(d)	(e)	(f)	
5	(a)	(b)	(c)	(d)	(e)	(f)	
6	(a)	(b)	(c)	(d)	(e)	(f)	
Total (Out of marks)							

Signature of Assessor / Cluster Mentor : _____

Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE
	Q.1:	
	A:	**Data Structure:-**
		When data is very huge or in large amount we need to organize it properly, so that we can access it efficiently. The mechanism which is used for this is called as data structure.
		Types of data structure:-
		a. Linear data structure:-
		If data is arranged in sequential order then it is called as Linear data structure.
		ex. Stack, Queue, Linked list
		b. Non-linear data structure:-
		If data is arranged in hierarchical manner then it is called as Non-linear data structure.
		ex. Tree, Graph.
	B.	**Graph traversal techniques:-**
		a. DFS. (Depth First Search).
		Algorithm:-
		1. Push starting node A into stack.
		2. Repeat step 3 and 4 until stack is empty.
		3. Pop the top node i.e. 'n' and process it.
		4. Push all the neighbors of 'n' into stack which are not visited.
		5. Stop.

Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE

b. BFS. (Breadth First Search)

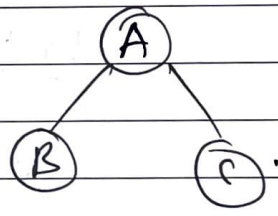
Algorithm:-

1. Insert starting node 'A' into queue
2. Repeat step 3 and 4 until queue is empty.
3. Delete the first node i.e. 'n' and process it.
4. Insert all the neighbours of 'n' into queue which are not visited.
5. Stop.

Q. Tree traversal techniques.

a. Inorder traversal.

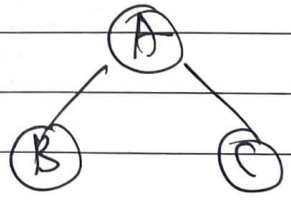
```
void inorder (node *t)
{
  if (t!=NULL)
  {
    inorder (t->left);
    printf ("%c", t->data);
    inorder (t->right);
  }
}
```



Inorder traversal :- B-A-C.

b. Preorder traversal

```
void preorder (node *t)
{
  if (t!=NULL)
  {
    printf ("%c", t->data);
    preorder (t->left);
    preorder (t->right);
  }
}
```



Preorder traversal :- A-B-C.

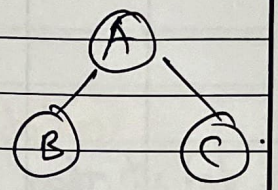
Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE

Q: Postorder traversal

```
void postorder (node *t)
{
  if (t!=NULL)
  {
    postorder (t->left);
    postorder (t->right);
    printf("%c", t->data);
  }
}
```

Postorder traversal:



B-C-A.

Q.2.

A. M-N*K/(G+T*S)-Q*S+D

Input	Stack	Postorder
M		M
-	-	M
N	-	MN
*	-*	MN
K	-*	MNK
/	-/	MNK*
C	-/C	MNK*
G	-/C	MNK*
+	-/C+	MNK*
T	-/C+	MNK*T
*	-/C+*	MNK*T
S	-/C+*	MNK*TS
)	-/	MNK*TS*+
-	-	MNK*TS*+/-
Q	-	MNK*TS*T/-Q

Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE

* — * MNK*TS*+/-@.
S — * MNK*TS*+/-@S.
+ + MNK*TS*+/-@S* -
D. + MNK*TS*+/-@S*-D

Postfix expression : MNK*TS*+/-@S*-D+

b. Linear Probing.

$h(k,i) = (k+i) \% N$.

25 70 80 10 54 20 37 30 50.

Solution:

Assume N = 11.

1. $h(25,0) = (25+0) \% 11 = 3$.
2. $h(70,0) = (70+0) \% 11 = 4$.
3. $h(80,0) = (80+0) \% 11 = 3 \rightarrow$ Collision.
   $h(80,1) = (80+1) \% 11 = 4 \rightarrow$ Collision.
   $h(80,2) = (80+2) \% 11 = 5$
4. $h(10,0) = (10+0) \% 11 = 10$.
5. $h(54,0) = (54+0) \% 11 = 10 \rightarrow$ Collision
6. $h(54,1) = (54+1) \% 11 = 0$
7. $h(20,0) = \cdots\cdots 9$
8. $h(37,0) = (37+0) \% 11 = 4 \rightarrow$ Collision
   $h(37,1) = (37+1) \% 11 = 5 \rightarrow$ Collision
   $h(37,2) = (37+2) \% 11 = 6$.

54.	0
	1
	2
25.	3
70.	4
80.	5
37.	6
50.	7
30.	8
20.	9
10.	10.

Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

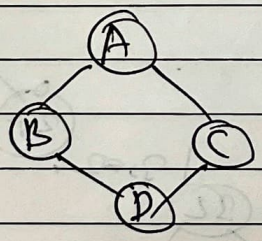
Space for Marks	Question No.	START WRITING HERE
	C:	**Representation of Graph Data Structure**

A graph can be represented in computer memory using following 2 ways:

a. Sequential representation (Array representation).

In this we represent a graph with the help of square matrix, having same order as that of number of vertices. Such matrix is called as "Adjacency matrix" and the matrix is formed as:

A_{ij} = 1 → If edge between i & j.
0 → If No edge between i & j.



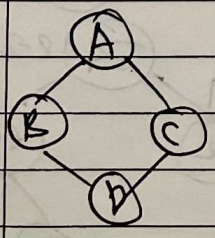
$A =$

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

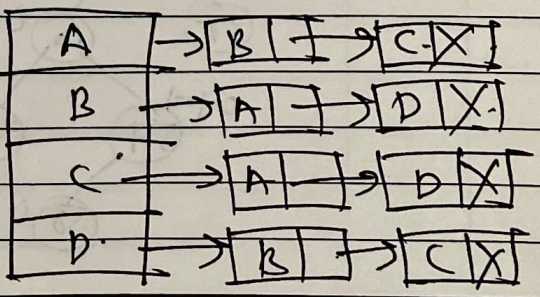
b. Linked Representation (Adjacency list).

Here a graph can be represented using linked list. A linked list contains different headers (start) same as that of different vertices of graph.

Adjacency list.



A	B C.
B	A D.
C	A D.
D	B C.



Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

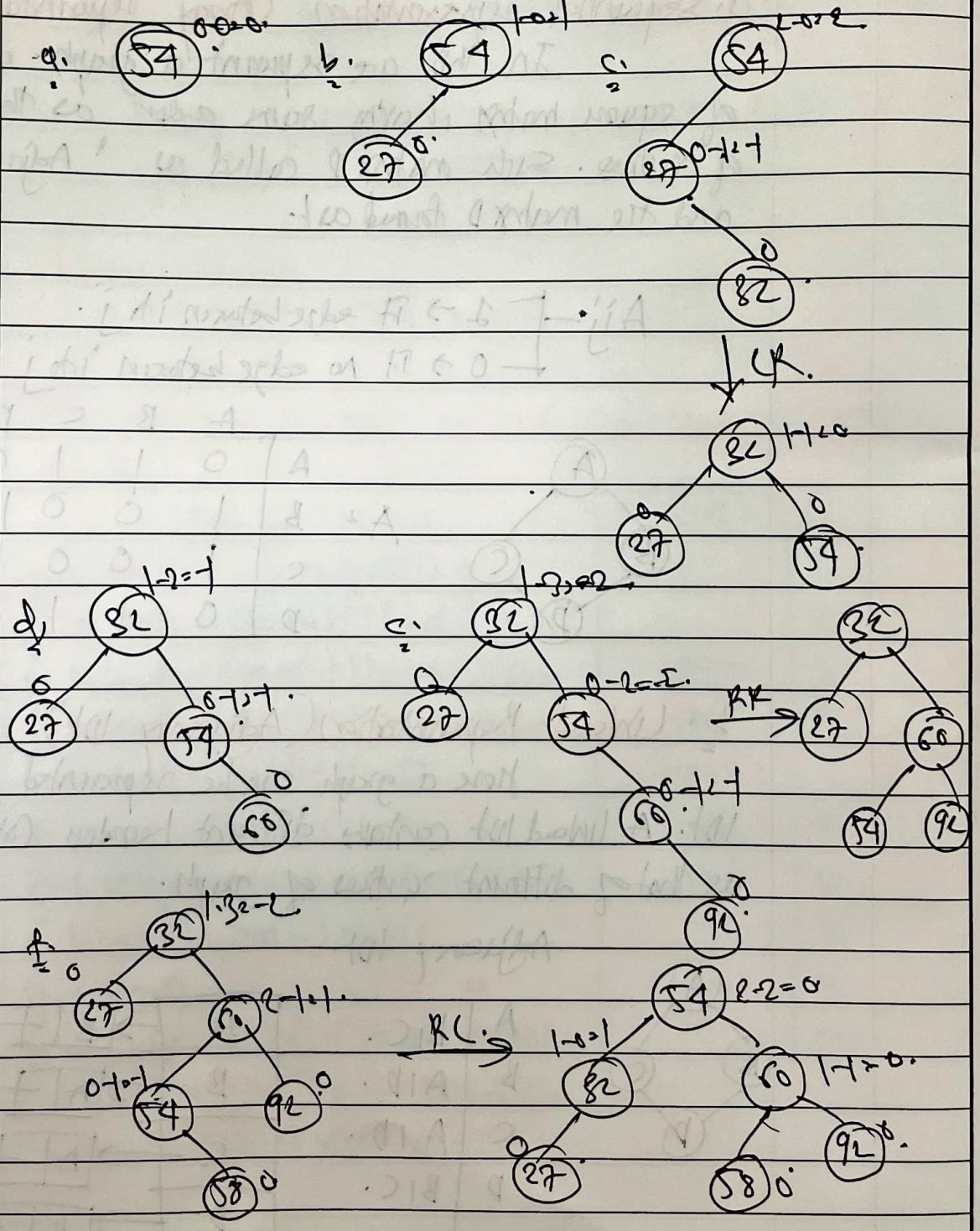
Space for Marks	Question No.	START WRITING HERE

Q.8.

A.

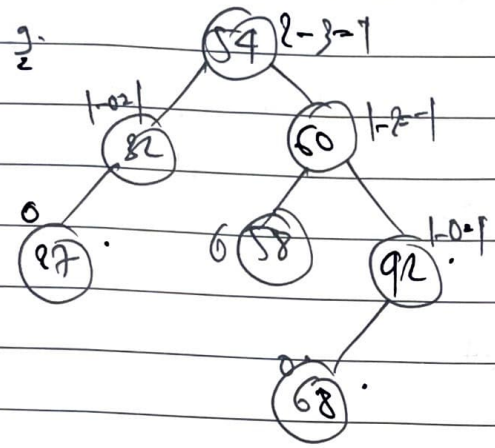
AVL Tree.

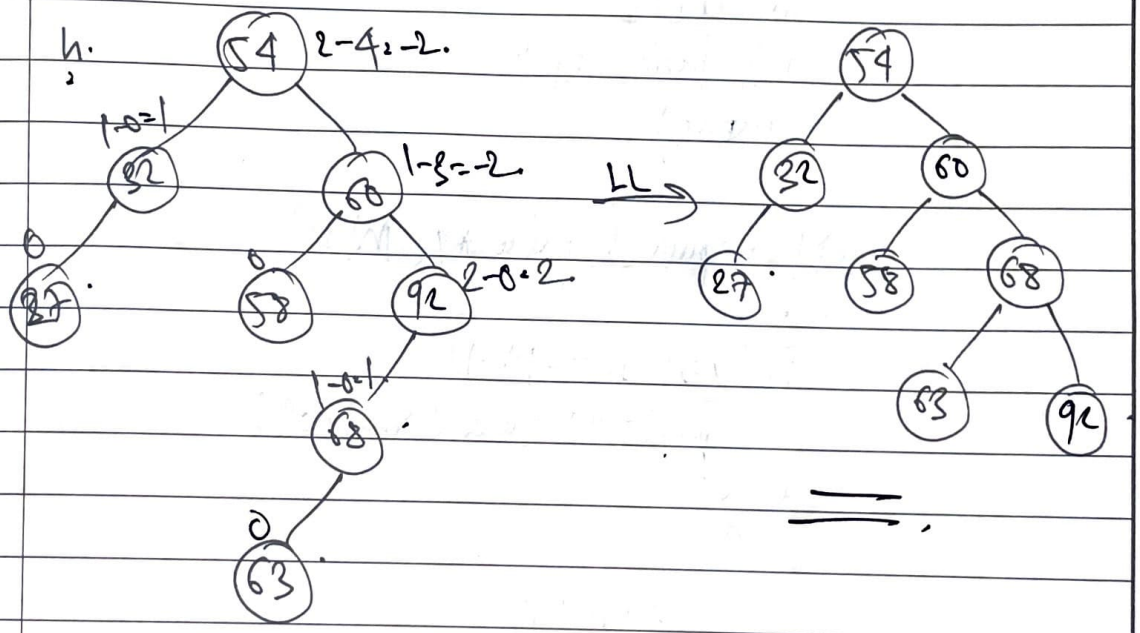
54 27 32 78 60 92 58 68 63



Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE





Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE
	B.	Simple (Linear) Queue using array.

```
#include <stdio.h>
#define N 10.
typedef struct queue
{
  int a[N];
  int front, rear;
} queue;

void enqueue (queue *q, int x)
{
  if(q->rear == N-1)
    printf("Queue Overflow");
  else
  {
    q->rear++;
    q->a[q->rear] = x;
  }
}
int Isempty (queue *q)
{
  if(q->front >= q->rear+1)
    return 1;
  else
    return 0;
}
```

Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE

```
int dequeue (queue *q)
{
  int x;
  if (isempty (q))
      return -1;
  else
    {
      x = q->a[q->front];
      q->front ++;
      return x;
    }
}

void display (queue *q)
{
  int i;
  if (isempty (q))
     printf ("\n Queue is empty");
  else
    {
      for (i=q->front; i<q->rear; i++)
        {
          printf ("\n%d", q->a[i]);
        }
    }
}
```

Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE

```
int main()
{
 int ch, x;
 queue q;
 q.front = 0;
 q.rear = -1;
 while(1)
 {
printf("\n\nMenu\n1.Enqueue\n2.Dequeue\n3.Display
      \n4.Exit\n Enter choice:");
scanf("%d", &ch);
if(ch==4)
   break;
switch(ch)
{
case 1:
 {
printf("\n Enter element to be inserted:");
 scanf("%d", &x);
enqueue(&q, x);
}
break;
case 2:
 {
  x = dequeue(&q);
  if(x==-1)
      printf("\n Queue underflow");
  else
      printf("\n Deleted element = %d", x);
}break;
```

```
case 3:
 {
  display(&q);
 }
 break;
default:
 {
  printf("\n Invalid choice");
 }
}
}
return 0;
```

Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE
	Q.4,	
	a.	Huffman Encoding!.

"MAHARASHTRA"

1. Frequency Table.

Symbol	Frequency.
M	1
A	4
H	2
R	2
S	1
T	1

2- Priority Queue.

(M11).

(M11) (A14).

(M11) (H12) (A14).

(M11) (H12) (R12) (A14).

(M11) (S11) (H12) (R12) (A14).

(M11) (S11) (T11) (H12) (R12) (A14).

Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE

3. Merging of nodes.

(M|1) · (S|1) (T|1) (H|2) (R|2) (A|4) ·

(T|1) (H|2) (R|2) (MS|2) (A|4) -

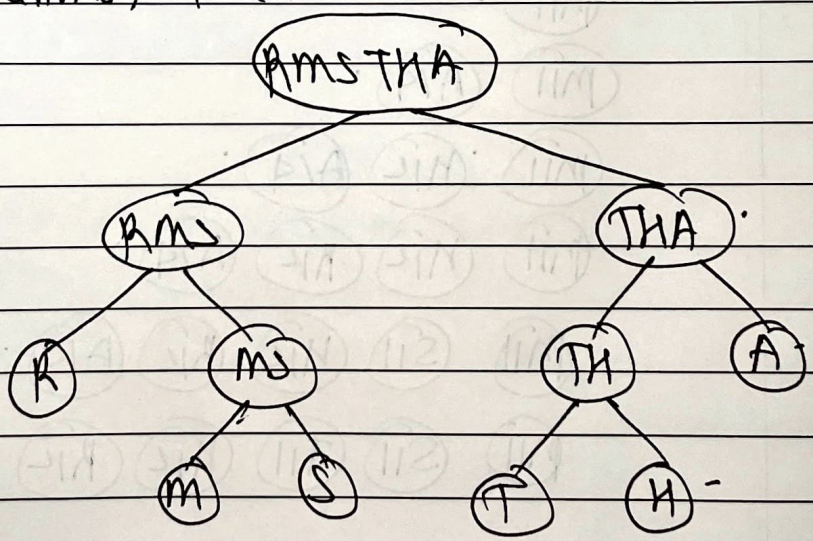
(R|2) (MS|2) (TH|3) · (A|4) -

(TH|3) (A|4) (RMS|4) -

(RMS|4) (THA|7) ·

(RMSTHA|11)

4. Huffman Tree.



Symbol	Huffman code
M	010 ·
A	11
H	101
R	00
S	011
T	100 ·

Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE

8.
=

Start using linked list.

```
#include <stdio.h>
#include <stdlib.h>
typedef struct node
{
    int data;
    struct node *next;
} node;

typedef struct LL
{
    node *start;
} LL;

void display (LL *l)
{
    node *p;
    if (l->start == NULL)
    {
        printf("\n List is empty");
    }
    else
    {
        p = l->start;
        while (p != NULL)
        {
            printf("\n %d", p->data);
            p = p->next;
        }
    }
}
```

Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE

```
void push(LL *l, int x)
{
 node *nearc;
 nearc = (node*)malloc(sizeof(node));
 nearc->data = x;
 nearc->next = NULL
 if(l->start == NULL)
  {
   l->start = nearc;
  }
  else
   {
    nearc->next = l->start;
    l->start = nearc;
   }
}

void pop(LL *l)
 {
  node *p;
  if(l->start == NULL)
  {
   printf("\n Deletion not possible");
  }
  else
   {
    p = l->start;
    l->start = l->start->next;
    free(p);
   }
}
```

Total Marks of Question no.		Examiner	
		Moderator	
		Re-Assessor	

Space for Marks	Question No.	START WRITING HERE

```
int main()
{
 int choice;
 ((*l;
 l->start = NULL;
 while(1)
  {
 printf("\n 1- Push \n 2- Pop \n 3- Display \n 4- Exit \n
            Enter choice :");
 scanf("%d", &ch);
 if(ch >= 4)
      break;
 switch(ch)
 {
 case 1:
  {
 printf("\n Enter element to
          be inserted = ");
 scanf("%d", &x);
  push(&l, x);
  display(&l);
 }
 break;

case 2:
   {
    pop(&l);
   display(&l);
   }
break;
```

```
case 3:
   {
    display(&l);
   }
   break;
default:
   {
    printf("\n Invalid
              choice !!");
   }
  }
 }
 return 0;
}.
```